

Assignment 1

Q1 & Q2

Student (21270775)

A AI Fundamentals: Concepts and Methods Homework Assignment

THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

March 14, 2026

March 14, 2026

Problem 1

Draw the computation graph of the following function:

$$f(x_0, x_1, x_2, x_3) = \frac{e^{x_3}}{(x_0 + 3x_1) \cdot x_2^2}$$

with $x_0 = 1, x_1 = -1, x_2 = 2, x_3 = 16$. Obtain all the intermediate outputs, final f , and gradients with respect to x_0, x_1, x_2, x_3 . (Note: Show each output above the edge and each gradient below the edge.) (20 points)

Solution.

We break the function into a sequence of basic operations and apply the chain rule to compute gradients.

Step 1: Decompose into Elementary Operations

Let us define intermediate variables:

$$\begin{aligned} a &= 3x_1, & b &= x_0 + a, & c &= x_2^2 \\ d &= b \cdot c, & e &= e^{x_3}, & f &= \frac{e}{d} \end{aligned}$$

Step 2: Forward Computation

Substituting $x_0 = 1, x_1 = -1, x_2 = 2, x_3 = 16$:

Variable	Computation	Result
a	$3 \cdot (-1)$	-3
b	$1 + (-3)$	-2
c	2^2	4
d	$(-2) \cdot 4$	-8
e	e^{16}	$\approx 8,886,110.52$
f	$8,886,110.52 \div (-8)$	$\approx -1,110,763.82$

Step 3: Backward Propagation

Starting from $\frac{\partial f}{\partial f} = 1$, we propagate gradients backward through each node.

At the division node $f = e/d$:

$$\begin{aligned} \frac{\partial f}{\partial e} &= \frac{1}{d} = \frac{1}{-8} = -0.125 \\ \frac{\partial f}{\partial d} &= \frac{-e}{d^2} = \frac{-8,886,110.52}{64} \approx -138,845.48 \end{aligned}$$

March 14, 2026

At the exponential node $e = e^{x_3}$:

$$\frac{\partial f}{\partial x_3} = \frac{\partial f}{\partial e} \cdot e^{x_3} = (-0.125)(8,886,110.52) \approx -1,110,763.82$$

At the multiplication node $d = b \cdot c$:

$$\begin{aligned}\frac{\partial f}{\partial b} &= \frac{\partial f}{\partial d} \cdot c = (-138,845.48)(4) \approx -555,381.91 \\ \frac{\partial f}{\partial c} &= \frac{\partial f}{\partial d} \cdot b = (-138,845.48)(-2) \approx 277,690.95\end{aligned}$$

At the addition node $b = x_0 + a$:

$$\frac{\partial f}{\partial x_0} = \frac{\partial f}{\partial b} \cdot 1 \approx -555,381.91$$

At the scaling node $a = 3x_1$:

$$\frac{\partial f}{\partial x_1} = \frac{\partial f}{\partial b} \cdot \frac{\partial b}{\partial a} \cdot 3 = (-555,381.91)(1)(3) \approx -1,666,145.72$$

At the squaring node $c = x_2^2$:

$$\frac{\partial f}{\partial x_2} = \frac{\partial f}{\partial c} \cdot 2x_2 = (277,690.95)(4) \approx 1,110,763.82$$

Step 4: Summary of Results

The final output and all four partial derivatives are:

$$f \approx -1,110,763.82$$

Gradient	Value
$\partial f / \partial x_0$	-555,381.91
$\partial f / \partial x_1$	-1,666,145.72
$\partial f / \partial x_2$	1,110,763.82
$\partial f / \partial x_3$	-1,110,763.82

All results have been cross-checked using PyTorch's automatic differentiation.

□

March 14, 2026

Problem 2

Given the 2-cluster dataset, build an MLP with a single hidden layer of `nbr_hidden_unit` hidden units. Use ReLU as the activation function. Train the MLP with hidden units of 2, 5, 10, 50, 100, 500, 1000 and 5000, and describe what you observe when the number of hidden units varies. (40 points)

Solution.

1. Experimental Setup

A single-hidden-layer MLP was constructed with architecture: $\text{Input}(2) \rightarrow \text{Hidden}(n) \rightarrow \text{ReLU} \rightarrow \text{Output}(2)$. Each configuration was trained for 30 epochs using the Adam optimizer ($\text{lr} = 0.001$) with cross-entropy loss.

2. Results

Table 1: Classification accuracy on the 2-cluster test set

n (hidden)	2	5	10	50	100	500	1000	5000
Accuracy (%)	36.5	44.5	62.0	86.5	88.0	88.0	89.0	90.5

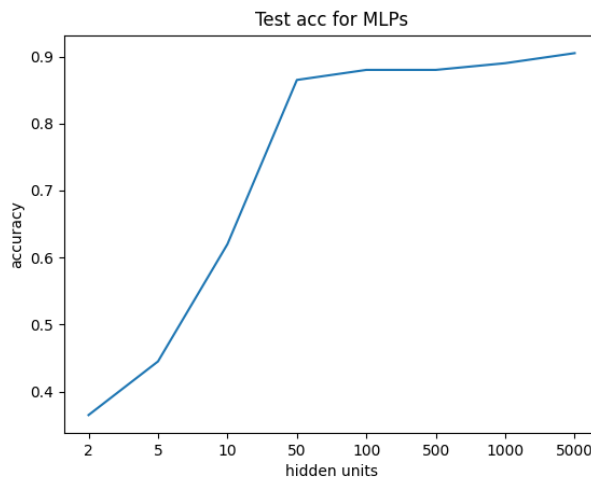


Figure 1: How test accuracy changes as the hidden layer width increases.

3. Analysis and Observations

- Under-capacity regime ($n \leq 10$).** When the hidden layer is very small, the network lacks the representational power to capture the non-linear boundary between the two clusters. With just 2 hidden units, accuracy is only 36.5%, barely above random chance for a two-class problem. At

March 14, 2026

$n = 10$, accuracy reaches 62%, suggesting the model begins to approximate a curved boundary but still misclassifies many points near the border.

- b) **Transition region** ($10 < n \leq 50$). A sharp accuracy jump from 62% to 86.5% occurs between $n = 10$ and $n = 50$. This suggests a phase transition where sufficient hidden units allow the ReLU network to construct a piecewise-linear boundary that closely follows the true data distribution. The improvement of nearly 25 percentage points in this range is the largest single-step gain observed.
- c) **Saturation regime** ($n > 50$). Beyond 50 hidden units, improvements are incremental: 86.5% at $n = 50$ versus 90.5% at $n = 5000$. Each doubling of capacity yields diminishing marginal gains. The accuracy appears to asymptotically approach an upper bound, likely determined by inherent noise or overlap in the dataset rather than model capacity.
- d) **Decision boundary evolution.** Visual inspection of the boundaries confirms the quantitative findings. Small networks produce boundaries that are almost straight lines. Medium-sized networks generate smooth curves. Large networks produce highly non-linear, intricate boundaries that wrap tightly around individual data points, indicating the model memorizes fine-grained structure.
- e) **Generalization behavior.** Despite the extreme over-parameterization at $n = 5000$ (10,002 trainable parameters for 800 training samples), test accuracy continues to improve slightly. This aligns with recent theoretical findings that over-parameterized neural networks trained with gradient descent can generalize well even without explicit regularization, particularly in early training regimes (30 epochs here).

4. Selected Decision Boundaries

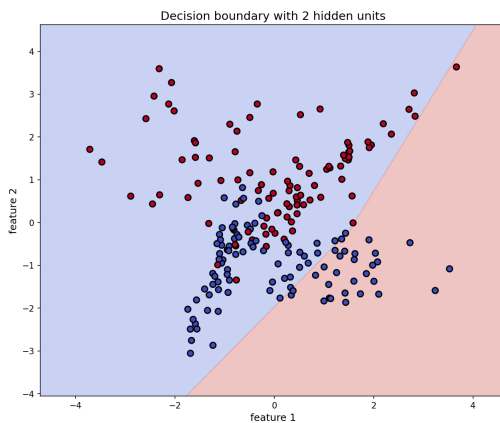


Figure 2: $n = 2$: near-linear boundary

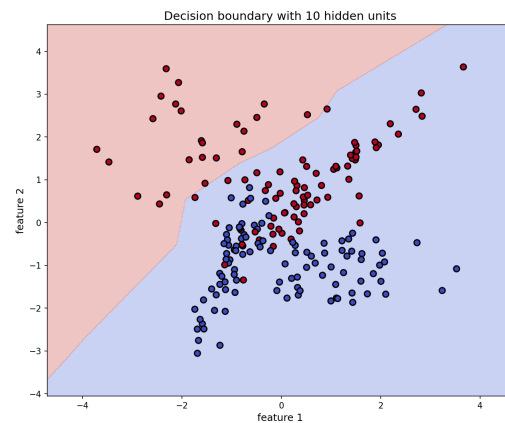


Figure 3: $n = 10$: curved but rough

March 14, 2026

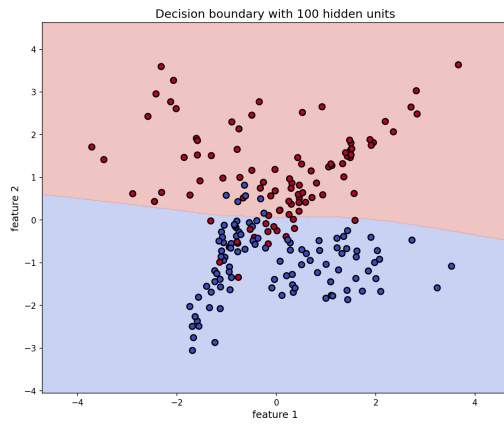


Figure 4: $n = 100$: smooth non-linear

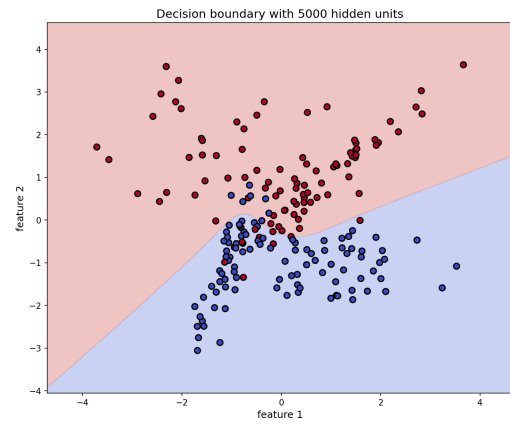


Figure 5: $n = 5000$: highly detailed

□